

Laboratorio di Programmazione

Lezione 11: Ricorsione

1	Funzioni ricorsive	2
2	Ristampa riga per riga.....	3
3	Ristampa riga per riga, invertito	4
4	Ristampa testo, invertito.....	5
5	Prodotto interno.....	6
6	Divisione tra interi.....	7
7	Sottoinsiemi.....	8
8	Permutazioni	9
9	Determinante di matrice.....	10
10	Qual è l'output?	11
11	Qual è l'output?	12
12	Qual è l'output?	13
13	Qual è l'output?	14

1 Funzioni ricorsive

Progettate, scrivete, e testate implementazioni **ricorsive** delle seguenti funzioni:

- Somma(n int) int che calcola $\sum_{i=1}^n i$ (se $n \leq 0$ la funzione ritorna 0).
- Prodotto(n int) int che calcola $\prod_{i=1}^n i$ (se $n \leq 0$ la funzione ritorna 1).
- SommaPot(n int, k int) int che calcola $\sum_{i=1}^n i^k$ (se $n \leq 0$ la funzione ritorna 1).
- SommaSeq(S []float64) float64 che calcola la somma degli elementi di S .
- Max(S []float64) float64 che ritorna il massimo di S .
- Media(S []float64) float64 che ritorna la media di S .
- InvertiStringa(s string) string che calcola l'inversa di s .
- Palindroma(s string) bool che decide se s è palindroma.
- Cerca(S []int, x int) int che ritorna il più piccolo $i \geq 0$ tale che $S[i]=x$, oppure -1 se x non compare in S .

Ricordate: la ricorsione prevede sempre un **caso base** e un **caso induttivo**.

2 Ristampa riga per riga

Scrivere un programma che legga da standard input un testo arbitrario, anche su più righe, terminato da EOF. Il programma deve stampare ogni riga, immediatamente dopo che è stata letta. Per fare questo, implementate in maniera **ricorsiva** una funzione:

```
LeggiEStampa(scanner *bufio.Scanner)
```

che, appunto, legge ogni riga e la ristampa. Il main quindi sarà semplicemente questo:

```
1 func main() {  
2     scanner := bufio.NewScanner(os.Stdin)  
3     LeggiEStampa(scanner)  
4 }
```

Esempio d'esecuzione:

```
$ go run righe.go  
Questa è una riga  
Questa è una riga  
Seconda riga in input  
Seconda riga in input  
Ogni riga viene stampata immediatamente  
Ogni riga viene stampata immediatamente
```

```
Anche le righe vuote sono stampate  
Anche le righe vuote sono stampate
```

3 Ristampa riga per riga, invertito

Adattare la soluzione dell'esercizio precedente nel seguente modo:

- il programma, prima di stampare, legge tutto l'input
- l'input è poi stampato invertendo l'ordine delle righe (dall'ultima alla prima)

Come prima, il main è semplicemente:

```
1 func main() {  
2     scanner := bufio.NewScanner(os.Stdin)  
3     LeggiEStampa(scanner)  
4 }
```

Esempio d'esecuzione:

```
$ go run inverti_righe.go  
Questa è una riga  
Seconda riga in input  
Ogni riga viene stampata in un secondo momento
```

```
Anche le righe vuote sono stampate  
Anche le righe vuote sono stampate
```

```
Ogni riga viene stampata in un secondo momento  
Seconda riga in input  
Questa è una riga
```

4 Ristampa testo, invertito

Adattare la soluzione dell'esercizio precedente in modo che il programma stampi il testo intero, invertito (cioè dall'ultimo carattere al primo). A questo scopo riutilizzate (i) la funzione `InvertiStringa` dell'esercizio 1 e (ii) la funzione `LeggiEStampa` dell'esercizio 3.

Esempio d'esecuzione:

```
$ go run inverti_testo.go
```

```
Questa è una riga
```

```
Seconda riga in input
```

```
Anche le righe vuote contano
```

```
onatonc etouv ehgir el ehcnA
```

```
tupni ni agir adnoceS
```

```
agir anu è atseuQ
```

5 Prodotto interno

Adattate la soluzione all'esercizio del prodotto interno tra due vettori (Lab 9 - Array e Slice) implementando una funzione ricorsiva:

```
1  ProdottoInterno(u, v []float64) float64
```

Esempio d'esecuzione:

```
$ go run prodotto_interno.go
-1.0 4.0 2.0 7.0
 2.0 3.0 -8.0 9.0
57.00
```

6 Divisione tra interi

Scrivere un programma che legga da riga di comando due interi m, n e calcoli il quoziente q ed il resto r della divisione intera tra m ed n . Ricordate che, per definizione, il quoziente q è il più grande intero che soddisfa $m = n \cdot q + r$ per qualche $r \geq 0$, e tale $r \geq 0$ è appunto il resto. A tale scopo implementate ricorsivamente una funzione:

```
1  Dividi(m, n int) (q, r int)
```

che, ovviamente, restituisce quoziente e resto della divisione di m per n .

Suggerimento: notate che, se $m \geq n$, allora

$$\begin{aligned} q &= q' + 1 \\ r &= r' \end{aligned}$$

dove q', r' sono quoziente e resto della divisione di $m - n$ per n .

Esempio d'esecuzione:

```
$ go run divisione.go 15 2  
q = 7, r = 1
```

```
$ go run divisione.go 12 3  
q = 4, r = 0
```

```
$ go run divisione.go 9 4  
q = 2, r = 1
```

7 Sottoinsiemi

Scrivere un programma che stampa tutti i sottoinsiemi dell'insieme di stringhe (parametri) passate da riga di comando. Per esempio:

```
$ go run sottoinsiemi.go a b c
{c} {b} {b,c} {a} {a,c} {a,b} {a,b,c}
```

A tale scopo implementate in maniera **ricorsiva** una funzione

```
Sottoinsiemi(X []string) [][]string
```

che restituisce l'elenco dei sottoinsiemi di X , dove ogni insieme è una slice di stringhe.

Suggerimento. Sia $\mathcal{P}(X)$ l'insieme dei sottoinsiemi di X (anche detto *insieme delle parti*). Fissato qualunque elemento $x \in X$, e definito $X' = X \setminus \{x\}$, è immediato vedere che:

$$\mathcal{P}(X) = \{A : A \in \mathcal{P}(X')\} \cup \{A \cup \{x\} : A \in \mathcal{P}(X')\}$$

In altre parole stiamo partizionando $\mathcal{P}(X)$ tra insiemi che non contengono x e insiemi che lo contengono, e i secondi si possono ottenere dai primi aggiungendo x . Pertanto ogni sottoinsieme A di X' “genera” i due sottoinsiemi A e $A \cup \{x\}$ di X . Usate questo fatto per calcolare $\mathcal{P}(X)$.

8 Permutazioni

Scrivere un programma che legga da riga di comando una stringa senza spazi e stampi tutte le permutazioni dei suoi simboli. A tale scopo implementate ricorsivamente una funzione `func Permutazioni(s string) []string` che restituisce l'elenco delle permutazioni di `s`.

Esempio d'esecuzione:

```
$ go run permutazioni.go ABC
ABC ACB BAC BCA CAB CBA
```

```
$ go run permutazioni.go ciao
ciao cioa caio caoi coia coai icaio icoa iaco iaoc ioca ioac acio acoi aico aioc
aoci aoic ocia ocai oica oiac oaci oaic
```

Suggerimento: considerate `Permutazioni(s[1:])`.

9 Determinante di matrice

Scrivere un programma che legga da standard input una matrice quadrata,

$$M = \begin{pmatrix} m_{11} & \dots & m_{1n} \\ \dots & \dots & \dots \\ m_{n1} & \dots & m_{nn} \end{pmatrix}$$

Il programma deve poi calcolare il determinante $\det M$ della matrice. Se $n = 1$, allora $\det M$ è l'unico elemento della matrice (cioè m_{11}). Se $n \geq 2$, allora $\det M$ può essere calcolato tramite il teorema di Laplace (https://it.wikipedia.org/wiki/Teorema_di_Laplace). Tale teorema dice che, per qualunque riga $i \in \{1, \dots, n\}$, vale:

$$\det M = \sum_{j=1}^n (-1)^{i+j} \cdot m_{i+j} \cdot \det M_{ij}$$

dove M_{ij} è la matrice ottenuta da M eliminando la riga i e la colonna j . (*Attenzione, stiamo usando indici che partono da 1*). A tale scopo implementate ricorsivamente una funzione `Determinante(M [][]float64) float64`.

Esempio d'esecuzione:

```
$ go run det.go
```

```
1 0 0 0
```

```
0 2 0 0
```

```
0 0 3 0
```

```
0 0 0 4
```

```
24.00
```

```
$ go run det.go
```

```
0.666667 -0.500000 0.200000 1.000000
```

```
1.000000 0.750000 -0.333333 0.000000
```

```
0.000000 0.400000 1.000000 -0.500000
```

```
-0.333333 0.000000 0.500000 1.000000
```

```
1.00
```

10 Qual è l'output?

```
1 package main
2 import ("fmt"; "strings")
3
4 func main() {
5     fmt.Printf("Funzione 'main' - Inizio\n")
6     Conteggio(3)
7     fmt.Printf("Funzione 'main' - Fine\n")
8 }
9
10 func Conteggio(n uint) {
11     rientro := strings.Repeat("\t", int(3-n))
12     fmt.Printf("%sFunzione 'Conteggio' - Inizio esecuzione per n = %d\n", rientro
13         , n)
14     if n == 0 {
15         fmt.Printf("%sPartenza!\n", rientro)
16     } else {
17         fmt.Printf("%s%d\n", rientro, n)
18         Conteggio(n - 1)
19     }
20     fmt.Printf("%sFunzione 'Conteggio' - Fine esecuzione per n = %d\n", rientro,
21         n)
22 }
```

11 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     conteggio(10)
6     fmt.Println("Partenza!")
7     fmt.Println()
8     conteggioAlternativo(10)
9     fmt.Println("Partenza!")
10 }
11 func conteggio(n uint) {
12     if n != 0 {
13         fmt.Println(n)
14         conteggio(n - 1)
15     }
16 }
17 func conteggioAlternativo(n uint) {
18     if n != 0 {
19         conteggioAlternativo(n - 1)
20         fmt.Println(n)
21     }
22 }
```

12 Qual è l'output?

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("La somma dei numeri da 0 a 10 è", Somma(10))
7 }
8
9 func Somma(n uint) uint {
10     if n == 0 {
11         return 0
12     } else {
13         return n + Somma(n-1)
14         // ... equivalente a:
15         // var somma_da_0_fino_a_n_meno_1 uint = Somma(n-1)
16         // return n + somma_da_0_fino_a_n_meno_1
17     }
18 }
```

13 Qual è l'output?

Quale proprietà viene testata?

```
1 package main
2 import "fmt"
3
4 func main() {
5
6     if P([]rune("1012982101")) {
7         fmt.Println("Proprietà soddisfatta.")
8     } else {
9         fmt.Println("Proprietà non soddisfatta.")
10    }
11
12    if P([]rune("1212982101")) {
13        fmt.Println("Proprietà soddisfatta.")
14    } else {
15        fmt.Println("Proprietà non soddisfatta.")
16    }
17
18 }
19
20 func P(sl []rune) bool {
21     fmt.Println(string(sl))
22     if len(sl) <= 1 {
23         return true
24     } else {
25         soddisfa := sl[0]==sl[len(sl)-1]
26         if soddisfa {
27             soddisfa = P(sl[1:len(sl)-1])
28         }
29         return soddisfa
30         // ... equivalente a:
31         // soddisfa := sl[0] == sl[len(sl)-1]
32         // return soddisfa && P(sl[1:len(sl)-1])
33     }
34 }
```
